INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Secure Development

Lab 1 Report

On

DevSecOps CI/CD

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 12.02.2026

Prerequisites Setup

I have completed the preparations to start a DevSecOps CI/CD lab on Ubuntu (VMware). My main goal was to create a stable environment where all the build, test, security scan and deploy related tasks can be done smoothly. Since the CI/CD pipeline is completely dependent on automation, it was very important to ensure that the necessary tools like Git, Curl, CA certificates and Node.js are already installed and updated.

First, I opened Terminal in Ubuntu using the Ctrl + Alt + T shortcut. Terminal is basically a Command Line Interface (CLI), through which one can run commands directly in the Linux shell. CLI is very important in DevOps and CI/CD work, because most of the automation, scripting, configuration and deployment tasks are performed from the command line. Therefore, using CLI is more efficient here than GUI.

Then I run the following command to install the basic tools.

```
hasib@gitlab-24:~$ sudo apt install -y git curl ca-certificates
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
curl is already the newest version (8.5.0-2ubuntu10.6).
ca-certificates is already the newest version (20240203).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 7 not upgraded.
```

Here I used *sudo* to get administrator privilege, because installing software requires root permission. apt is Ubuntu's package manager (Advanced Package Tool), through which software can be installed, updated or removed. The install subcommand indicates that I want to install a new package. I used the -y option so that it automatically gets “Yes” approval when asked for confirmation during installation.

This command installs or verifies Git, Curl and CA certificates. Git is a version control system, which is essential for source code management and pulling code in CI/CD pipelines. Curl is a command line HTTP client, which is used to download API calls or repository setup scripts. CA certificates ensure secure HTTPS communication and help in SSL certificate verification. After running the command, the output shows that the package list has been read from the system repository, the dependency tree has been created and the current state has been verified. Git, Curl and CA certificates were already installed in the latest version - this is confirmed. No new packages have been installed or upgraded, which proves that my environment was already prepared. It also warned about

an unnecessary dependency (*libllvm19*), which can be removed with *sudo apt autoremove*.

I then move on to installing Node.js 20, as we need to build a small web application in our DevSecOps pipeline, which requires the Node.js runtime. First, I run the following command.

```
hasib@gitlab-24:~$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
2026-02-11 13:11:29 - Installing pre-requisites
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 https://deb.nodesource.com/node_20.x nodistro InRelease
Hit:5 https://download.docker.com/linux/ubuntu noble InRelease
Get:6 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,741 kB]
Hit:7 https://packages.gitlab.com/gitlab/gitlab-ce/ubuntu noble InRelease
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:11 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:12 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:13 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,316 B]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:15 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:16 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 2,573 kB in 3s (929 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apt-transport-https is already the newest version (2.8.3).
ca-certificates is already the newest version (20240203).
curl is already the newest version (8.5.0-2ubuntu10.6).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
```

Here, the NodeSource repository setup script has been downloaded using Curl. The *-f* option causes a silent fail if there is a server error, -s shows less output, -S shows if there is an error, and *-L* redirects the follow. Using a pipe (|), the output of Curl is sent directly to the *sudo -E bash* - command, which executes the script in the Bash shell with root privileges. The -E option preserves the environment variable.

Analyzing the output, it can be seen that the NodeSource repository has been successfully added and system update information is shown. Then I installed Node.js with this part *install -y nodejs*. Here, it can be seen that Node.js version 20.20.0 was already installed and was present as the latest version.

Finally, I performed version verification by running the *node -v* and *npm -v* commands.

```
hasib@gitlab-24:~$ node -v
v20.20.0
```

```
hasib@gitlab-24:~$ npm -v
10.8.2
```

The output shows *v20.20.0*, which confirms that the Node.js runtime is properly installed. It also shows *npm version 10.8.2*. npm (Node Package Manager) is an essential tool for dependency management and project builds, which will be used in the CI/CD pipeline.

By successfully completing these steps, I have ensured that my Ubuntu Virtual Machine is fully ready to run the DevSecOps CI/CD lab. All the necessary tools are installed and updated, allowing for seamless pipeline configuration and application deployment in the next steps.

Task 1: Pipeline Foundation and Build

Create a repository and configure a pipeline that runs on pushes to the main branch. The pipeline must check out the code, install dependencies, build the project, and run tests. The run must fail if tests fail.

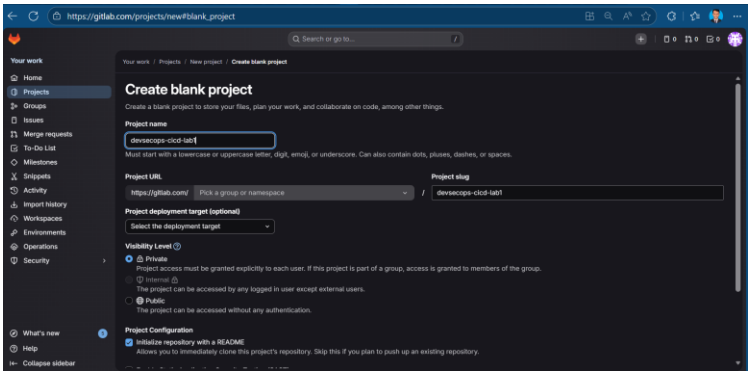
Document the trigger conditions, branch behavior, and how the build can be reproduced locally using a script or Docker.

Solution:

The purpose of this task is to create a project repository in GitLab and configure a Continuous Integration (CI) pipeline that automatically starts when a push is made to the main branch. The pipeline will step by step checkout the code, install dependencies, build the project, and run unit tests. If any test fails, the pipeline fails. It also demonstrates how to reproduce the build on a local machine and via Docker.

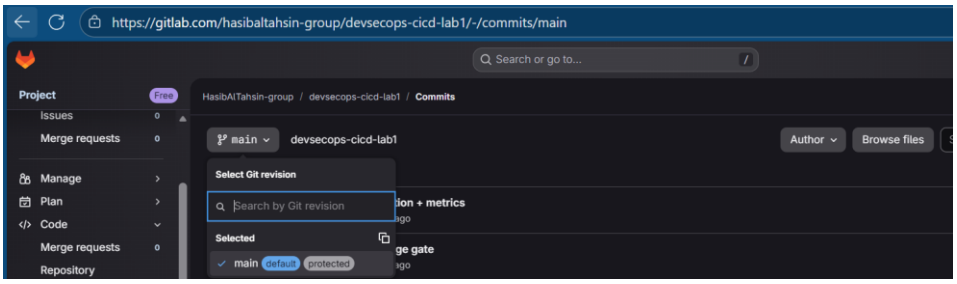
Creating a GitLab Repository:

A blank project named devsecops-cicd-lab1 is created using the GitLab web interface and the "Initialize with README" option is selected.



Checking the default branch main:

Then an initial commit is made and main is set as the default branch. It is verified from Branches in the Repository that main is the default branch. This setup is important, because later the CI pipeline only runs when pushing to the main branch.



Repo Clone:

The remote GitLab repository has been copied to the local Ubuntu VM using the following command. This command fetches all the commit history and files from the remote server, creates a local *.git* directory, and checkouts the main branch.

```
hasib@gitlab-24:~$ git clone https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
```

Then the following command was used to enter the project folder. This created a tracking relationship between the local and remote repositories.

```
hasib@gitlab-24:~$ cd devsecops-cicd-lab1
hasib@gitlab-24:~/devsecops-cicd-lab1$
```

Project Initialization:

The following command creates a default *package.json* file.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm init -y
```

The output "*Wrote to … package.json*" shows that the file was successfully created. It defines the project name, version, entry point (*src/server.js*), and three important scripts:

1. start: to start the server
2. test: to run tests with Jest
3. build: a declarative build command

I added express and *lodash* to dependencies and *jest* and *supertest* to *devDependencies.*

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm init -y
Wrote to /home/hasib/devsecops-cicd-lab1/package.json:

{
  "name": "devsecops-cicd-lab1",
  "version": "1.0.0",
  "description": "",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "test": "jest --runInBand",
    "build": "node -e \"console.log('build ok')\""
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1",
    "lodash": "^4.17.23"
  },
  "devDependencies": {
    "jest": "^30.2.0",
    "supertest": "^7.2.2"
  },
  "directories": {
    "doc": "docs",
    "test": "test"
  }
}
```

The following two commands installed the packages. The output "*audited … packages*" and "*found 0 vulnerabilities*" indicate that the install was successful and *npm audit* did not detect any known vulnerabilities.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm i express

up to date, audited 381 packages in 3s

70 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm i -D jest supertest

up to date, audited 381 packages in 3s

70 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

The directory structure src, test, scripts, reports, docs has been created.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ mkdir -p src test scripts reports docs
```

package.json Scripts Configuration:

Main and scripts fields in *package.json* have been rewritten using Node one-liners.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ node - <<'NODE'
const fs=require('fs');
const p=JSON.parse(fs.readFileSync('package.json','utf8'));
p.main="src/server.js";
p.scripts={start:"node src/server.js",test:"jest --runInBand",build:"node -e \"console.log('build ok')\""};
fs.writeFileSync('package.json', JSON.stringify(p,null,2));
console.log("package.json updated");
NODE
```

This confirms that *npm start*, *npm test,* and *npm run* build run the specified commands. The output "*package.json updated*" indicates a successful change.

Express Server Code:

An Express app is defined in *src/server.js. /* returns a JSON response *{ ok: true, message: "hello"* } in the root.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano src/server.js
```

Additionally, an intentionally insecure endpoint called */cmd* has been created, where shell commands can be executed via query parameters using *child_process.exec.* This is reserved for later SAST proofs.

```
  GNU nano 7.2                                                  src/server.js
const express = require("express");
const app = express();

app.get("/", (req, res) => res.json({ ok: true, message: "hello" }));

// Intentionally insecure endpoint for SAST evidence (remove later to pass gate)
app.get("/cmd", (req, res) => {
  const { exec } = require("child_process");
  exec(req.query.c || "echo safe", (err, stdout, stderr) => {
    if (err) return res.status(500).json({ err: String(err) });
    res.json({ out: stdout, err: stderr });
  });
});

module.exports = app;

if (require.main === module) {
  const port = process.env.PORT || 3000;
  app.listen(port, () => console.log(`listening on ${port}`));
}
```

The *if (require.main === module*) block starts the server on *port 3000*.

Unit Test:

A GET / request was sent using Jest and *Supertest* in the *test/app.test.js* file.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano test/app.test.js
```

The test verifies that the status code is *200* and *ok: true* is returned. This confirms that the basic endpoint of the app is working.

```
  GNU nano 7.2                                                    test/app.test.js
const request = require("supertest");
const app = require("../src/server");

describe("GET /", () => {
  it("should return ok", async () => {
    const res = await request(app).get("/");
    expect(res.statusCode).toBe(200);
    expect(res.body.ok).toBe(true);
  });
});
```

Local Execution Proof

After running the following command, the output shows "PASS test/app.test.js" and "*1 passed, 1 total*" indicating that all tests were successful. Jest does not return a non-zero exit code, meaning it is successful.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm test

> devsecops-cicd-lab1@1.0.0 test
> jest --runInBand

 PASS  test/app.test.js
  GET /
    ✓ should return ok (46 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.381 s
Ran all test suites.
```

The following command prints "*build ok*", confirming that the build script was executed.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm run build

> devsecops-cicd-lab1@1.0.0 build
> node -e "console.log('build ok')"

build ok
```

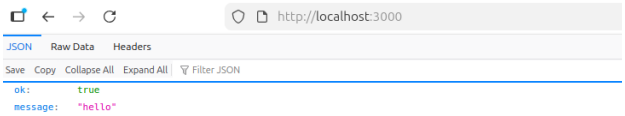
The output of "*listening on 3000*" in the command below shows that the server has been started successfully.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm start

> devsecops-cicd-lab1@1.0.0 start
> node src/server.js

listening on 3000
```

Opening *http://localhost:3000* in the browser yields *{ ok: true, message: "hello" },* which proves that the app is working properly.



.gitlab-ci.yml Explanation:

The workflow.rules block specifies that the pipeline will only run when *$CI_PIPELINE_SOURCE* is "*push*" and *$CI_COMMIT_BRANCH* is "*main*".

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano .gitlab-ci.yml
```

Pushing to any other branch will not run the pipeline. This ensures controlled branch behavior. *default.image: node:20-bullseye* indicates that each job will run in a Node 20 environment in the Docker image. The version is checked with *node -v* and *npm -v* in the *before_script* and a clean install is performed with *npm ci. npm ci* installs lockfile-based deterministic dependencies, which ensures reproducible builds. The build job runs *npm run build* and saves *package.json* and *package-lock.json* as artifacts. The test job runs *npm test*. If Jest fails any test, it returns a non-zero exit code and the GitLab job fails. This causes the entire pipeline to fail, which satisfies the task condition. GitLab Runner automatically checkouts the repository before each job starts, so a separate checkout command is not required.

```
  GNU nano 7.2                                                .gitlab-ci.yml *
stages:
  - build
  - test
  - security
  - container

workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "push" && $CI_COMMIT_BRANCH == "main"'
      when: always
    - when: never

default:
  image: node:20-bullseye
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths:
      - .npm/
  before_script:
    - node -v
    - npm -v
    - npm ci --cache .npm --prefer-offline
```

```
build:
  stage: build
  script:
    - npm run build
  artifacts:
    when: always
    paths:
      - package.json
      - package-lock.json
    expire_in: 14 days

test:
  stage: test
  script:
    - npm test

^G Help         ^O Write Out    ^W Where Is     ^K Cut          ^T Ex
^X Exit         ^R Read File    ^\ Replace      ^U Paste        ^J Ju
```

Commit and Push:

The *.gitlab-ci.yml* file is sent to the remote repository using the git add, git commit, and git push commands.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

The output shows "*main -> main*" and the push was successful. The GitLab CI pipeline is automatically triggered.

commit:

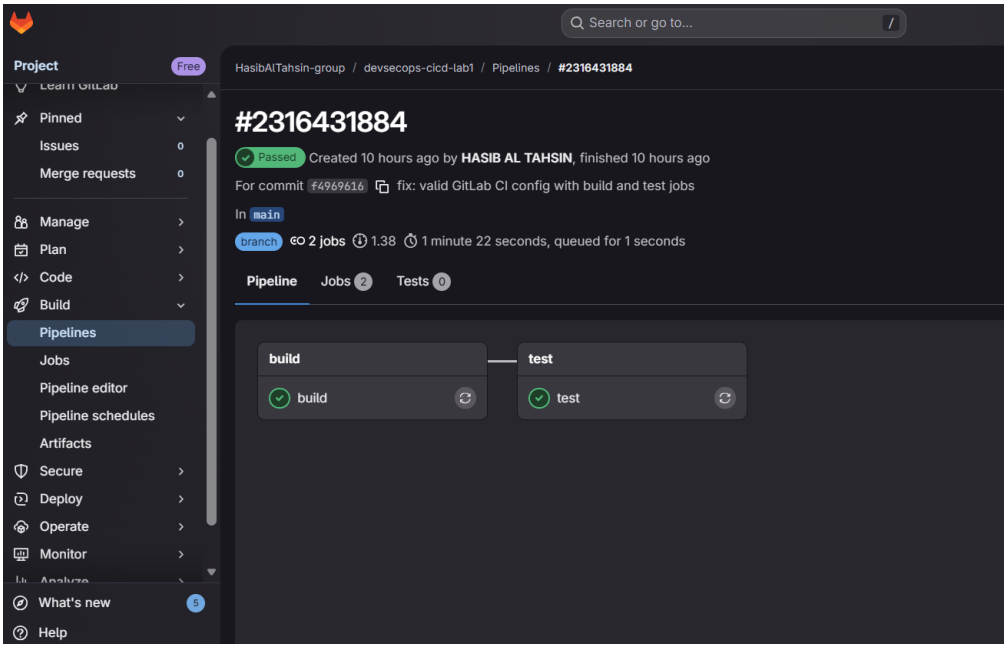
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: valid GitLab CI config with build and test jobs"
[main f496961] fix: valid GitLab CI config with build and test jobs
 1 file changed, 9 deletions(-)
```

push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 328 bytes | 328.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   718f0f9..f496961  main -> main
```

Pipeline Evidence:

In the GitLab web UI, by going from CI/CD to Pipelines, you can see that both the build and test jobs in the latest pipeline are passed. This proves that the pipeline has successfully completed execution.



Local reproduce:

In this step, a local script is created in which the "*dependency install then build then test*" steps of the GitLab CI pipeline are executed in the same order on the local machine with one click. This report proves that the pipeline can be reproduced not only in the GitLab runner but also in the local environment. The following command creates a new file named *scripts/local-ci.sh* and writes the script inside it.

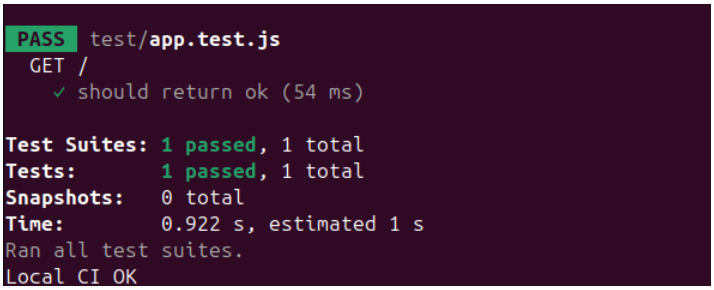
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ cat > scripts/local-ci.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
npm ci
npm run build
npm test
echo "Local CI OK"
EOF
```

Here, the *#!/usr/bin/env* bash line means that this file will be run with Bash. The *set -euo pipefail* part is very important for mimicking CI behavior: *-e* means that if any command fails, the script stops immediately, *-u* means that using an undefined variable will result in an error (prevents silent bugs), and *pipefail* means that even if any part of the pipeline fails, the overall failure will occur. Then *npm ci* is run so that the specific dependencies are installed cleanly according to *package-lock.json*. Then *npm run build* is run to verify whether the build step is successful or not. Then *npm test* is run to see if the unit test

passes or not. Then the *echo "Local CI OK"* line gives a human-readable success stamp meaning that all three steps have been completed correctly.

The script is made executable with the following command. To run a file on Linux, execute permission is required, so *+x* is set. Then *./scripts/local-ci.sh* is run to execute the script.







The *npm warn deprecated ...* lines at the beginning of the output are not errors, but warnings. This means that some of the installed dependencies contain old packages that are not being maintained or are old versions. However, these are not errors that can cause pipeline failures, because they are just informational warnings. Found 0 vulnerabilities means that no known vulnerabilities were found at that time according to the *npm audit*.

Then in the build step *> devsecops-cicd-lab1@1.0.0 build* shows that *npm run build* is running a script, and below it, the *node -e "console.log('build ok')"* command is executed. If build ok is printed, the build stage has passed.

Then in the test step *> jest --runInBand* means that Jest is running tests sequentially (not in parallel), which increases stability in the CI environment. *PASS test/app.test.js* means that the test file is successful, and should return ok shows that the specified assertion has passed. Test Suites: 1 passed means suite-level pass, *Tests: 1 passed* means test-case pass.

Time: ... means how long it took. Finally, *Local CI OK* shows that there were no errors till the end of the script and the local run like CI pipeline was also successful.

Then the following commands were used so that this reproduce evidence is permanently in the repository.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add scripts/local-ci.sh
```

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task1: local reproduce script"
[main 2ab3d8a] Task1: local reproduce script
 1 file changed, 6 insertions(+)
 create mode 100755 scripts/local-ci.sh
```

In the commit output, *create mode 100755* means the file permission is saved as executable (*755 = executable*). In the push output, *main -> main* shows that the local commit has successfully moved to the remote main branch, which also makes it possible to trigger the pipeline again in GitLab.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 413 bytes | 82.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   f496961..2ab3d8a  main -> main
```

Docker reproduce:

This part is optional for Task 1, but it is very strong evidence, because using Docker proves that the app is not a "run on my laptop" type, but rather reproducible in a clean container environment. That is, it is possible to build/run the same way even if there are OS/library differences.

The *FROM node:20-bullseye* in the Dockerfile means that the Debian Bullseye image with *Node.js 20* as the base environment has been taken, which is also consistent with the image choice of the CI runner.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano Dockerfile
```

```
  GNU nano 7.2                                                  Dockerfile *
FROM node:20-bullseye

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 3000
CMD ["node","src/server.js"]
```

WORKDIR /app means that the working directory /app is set inside the container. *COPY package*.json ./ First*, only the package file is copied so that Docker layer caching works well (if the dependency does not change, you do not have to run *npm ci* again). *RUN npm ci* means that a clean dependency is installed according to the lockfile which is the reproducibility core. Then the rest of the source code is copied with *COPY . . EXPOSE 3000* means that the container will use port 3000 (documentation + runtime mapping facility). *CMD ["node","src/server.js"]* means that the server will run with Node when the container is started.

A service has been created in *docker-compose.yml with services: app: , build: .* means that the image will be built by reading the Dockerfile from the current directory, and *ports: "3000:3000"* means that *port 3000* of the host machine will be mapped to *port 3000* of the container, so that the app can be viewed with a browser.

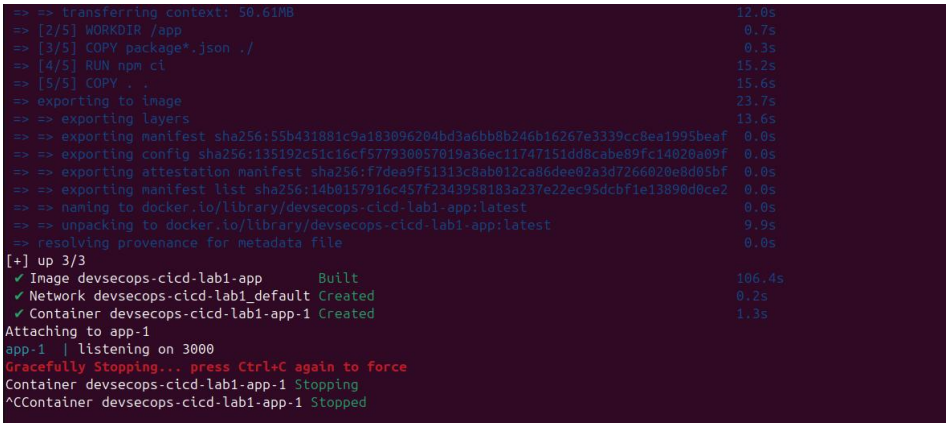
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano docker-compose.yml
```

```
  GNU nano 7.2                                                docker-compose.yml
services:
  app:
    build: .
    ports:
      - "3000:3000"
```

Running the following command will result in the output *[+] Building ... FINISHED* meaning the image has been built.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ docker compose up --build
```

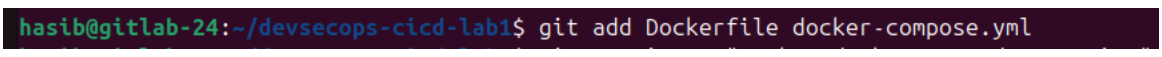




The load metadata and *FROM node:20-bullseye@sha256:...* parts show that the base image has been pulled/resolved from Docker Hub. The various sha256:... *MB* lines are the image layer downloaded/extracted. Then *[4/5] RUN npm ci* shows that the dependency has been installed during the container build. Finally, naming to *docker.io/library/devsecops-cicd-lab1-app:latest* means that the image has been named and the container has been started. Attaching to *app-1 and app-1 | listening on 3000* means that the container runtime has successfully started the server, which is the biggest evidence that the app is working even in a clean Docker environment. *Gracefully Stopping...* and finally Stopped means that the container has been stopped with Ctrl+C.

Then *git add*, *commit* and *push* have been done, so that the Docker reproduce option is documented in the repository.

add:



commit:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task1: docker reproduce option"
[main 9d98fb3] Task1: docker reproduce option
 2 files changed, 12 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 docker-compose.yml
```

push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 505 bytes | 252.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   2ab3d8a..9d98fb3  main -> main
```

The commit output shows the addition of a new file and the push output shows the update of the remote main, completing the evidence chain.

Task 2: Secrets and Credentials

Add a secrets scanning step to the pipeline and configure it to fail the run when hardcoded credentials are detected. Store all pipeline credentials in the CI platform's secret manager and ensure they never appear in logs.

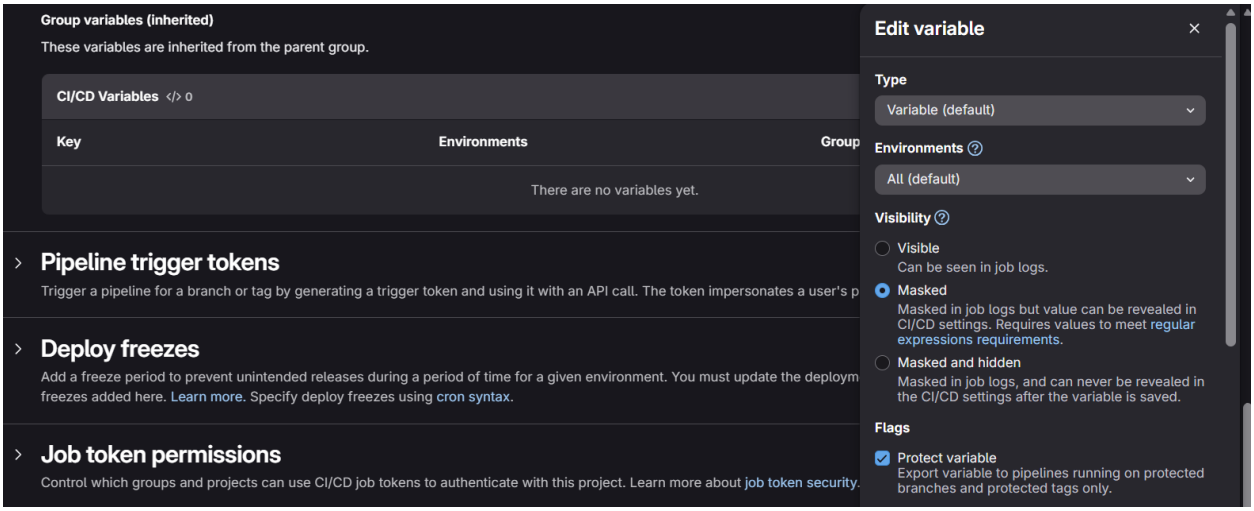
Create a temporary commit containing a fake secret to prove the gate works, then remove it and show a passing run.

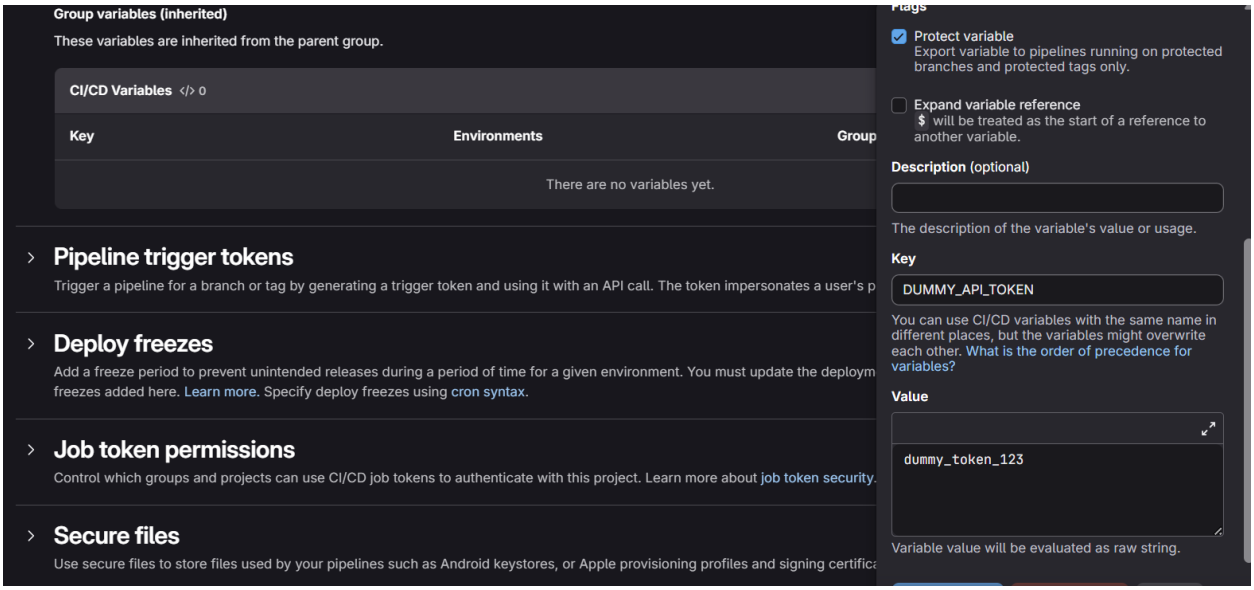
Tool choice: Secrets in code: GitLeaks, TruffleHog, or detect-secrets (or GitGuardian, SonarQube). Pipeline credentials: platform secret store (GitHub Secrets, GitLab variables, Jenkins credentials).

Solution:

Adding Variable to GitLab Secret Store:

First, I went to the GitLab project settings and opened Variables from CI/CD. From there, I added a dummy token using Add variable, *DUMMY_API_TOKEN* as Key and *dummy_token_123* as Value. Here, I enabled the "*Masked*" option, because if masked, even if this value is echoed/printed in the pipeline logs for some reason, GitLab masks it and does not show it as the real text. I kept the "*Protected*" option as optional, because if protected is given, this variable is only usable in protected branches/tags.





Adding *GitLeaks* Job to *.gitlab-ci.yml*:

Next I added *GitLeaks* job to *.gitlab-ci.yml* so that pipeline fails if there is any secret pattern inside the repo. I appended it with the below command.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ cat >> .gitlab-ci.yml <<'EOF'

gitleaks:
  stage: security
  image:
    name: zricethezav/gitleaks:latest
    entrypoint: [""]
  script:
    - mkdir -p reports
    - gitleaks detect --source . --report-format json --report-path reports/gitleaks.json --exit-code 1
  artifacts:
    when: always
    paths:
      - reports/gitleaks.json
    expire_in: 14 days
EOF
```

The advantage of appending this way is that the new job block is added right below without destroying the previous config and the possibility of YAML indentation errors is also reduced.

I named this job *gitleaks* and set stage: security, so that the scan runs in the security stage after the build/test is completed. I used *zricethezav/gitleaks:latest* as the image, which means that I did not have to install *gitleaks* separately on the runner's machine; the tool is ready from the Docker image. I set *entrypoint: [""],* because if many images have a default *entrypoint,* the GitLab runner script command may not execute properly; if the *entrypoint* is empty, the script lines are run directly through the shell.

In the script part, I first did *mkdir -p reports*, so that even if there is no reports/ folder to store the report, it is created. Then the main command was run. Where *--source* . means that the current repository will be scanned. A JSON report was created with *--report-format json*, and it was written to the *reports/gitleaks.json* file with *--report-path*. The most important is *--exit-code 1*. This will cause the command to return a non-zero exit code if a secret is detected, and the CI job will treat it as a failure and fail the pipeline. This is the gate behavior.

I used when: always in the artifacts section so that regardless of whether the job passes/fails, the *reports/gitleaks.json* file remains as an artifact. Even if it fails, the report can be downloaded as evidence. The artifact retention time is set to *expire_in: 14 days*.

Then, I staged the *.gitlab-ci.yml* file and committed it.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task2: add gitleaks secret scanning gate"
[main a843546] Task2: add gitleaks secret scanning gate
 1 file changed, 14 insertions(+)
```

The commit output shows that a new commit has been created in the main branch and 14 lines of new config have been added to *.gitlab-ci.yml* (this is the *gitleaks* job block). Then, the output that came when git push showed *main -> main*. That is, the remote main has been updated from local main.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 512 bytes | 512.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   9d98fb3..a843546  main -> main
```

Along with this, GitLab CI automatically triggered the pipeline (because it was pushed to main) and from now on, *gitleaks* scan has been added to the security stage of the pipeline.

FAIL Proof with Fake Secret Commit:

I intentionally created a fake secret file to prove that Gate is really working. I opened a new file with nano editor and wrote fake AWS credentials as below.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano src/secret_temp.js
```

```
  GNU nano 7.2                                                  src/secret_temp.js *
module.exports = {
  AWS_ACCESS_KEY_ID: "AKIA1234567890FAKEKEY",
  AWS_SECRET_ACCESS_KEY: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYFAKESECRET"
};
```

Since these are actually similar to AWS key patterns, secret scanners usually detect them. I then staged and committed this file.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: add security stage and gitleaks job"
[main 5245958] fix: add security stage and gitleaks job
 1 file changed, 1 insertion(+)
```

Output shows:

[main 5245958] … 1 file changed, 1 insertion(+)

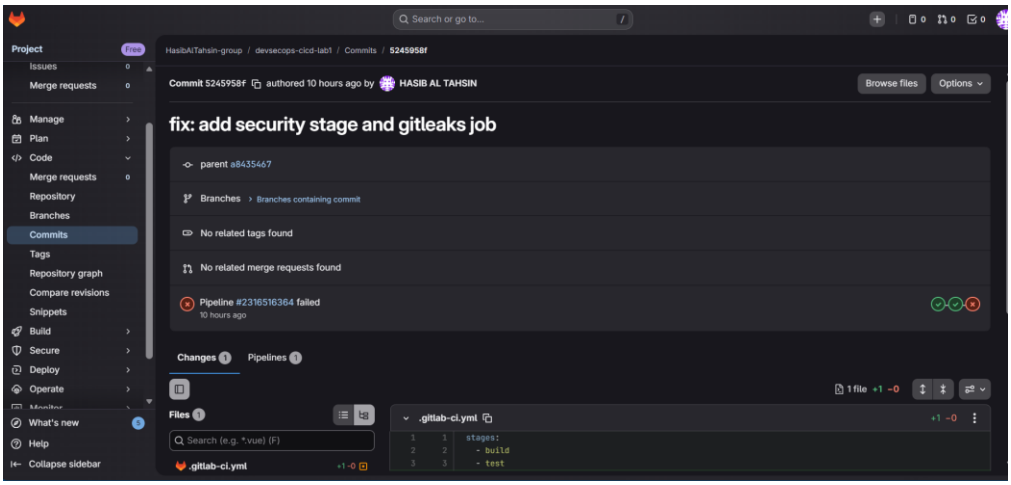
Then after git push, push output shows:

a843546..5245958 main -> main

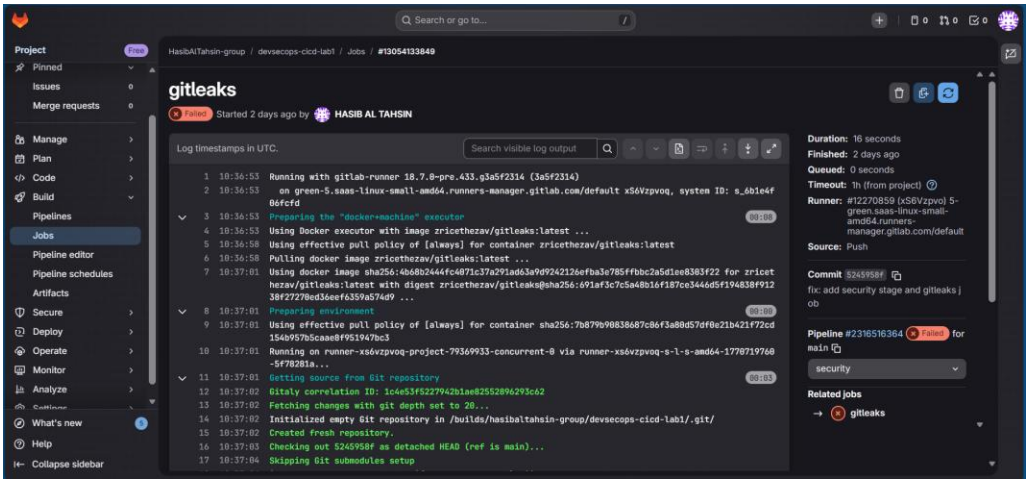
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 329 bytes | 329.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   a843546..5245958  main -> main
```

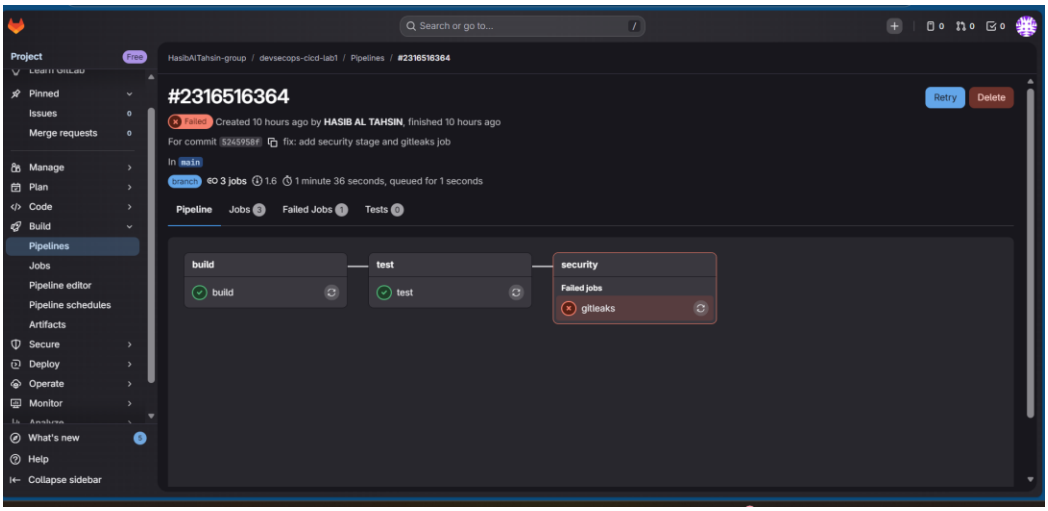
This means that the remote main has been updated from the previous commit to this new commit and as a result the pipeline has been run again. In this run, the GitLeaks job detects that fake secret pattern and FAILS the job.

Evidence: Pipelines - *gitleaks* job FAILED



Artifact: *reports/gitleaks.json*





Remove Secret (Security Gate Clean Verification):

In the previous step, a fake AWS secret was intentionally committed and as a result, the GitLab pipeline's *gitleaks* job failed. Now in this step, it has been proven that removing the secret makes the pipeline pass normally again. Another important thing here is that *$DUMMY_API_TOKEN* was never printed directly in the logs. The GitLab variable is stored as "*Masked*", so even if a job accidentally echoed it, the real value would not be visible in the logs.

First, the file was deleted from the repository using the following command.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git rm src/secret_temp.js
rm 'src/secret_temp.js'
```

The output shows that the file has been removed from the working directory and staging area. This file will no longer be present in the next commit.

The change is then committed using the following command.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task2: remove fake secret (expect PASS)"
[main 2e50352] Task2: remove fake secret (expect PASS)
 1 file changed, 4 deletions(-)
 delete mode 100644 src/secret_temp.js
```

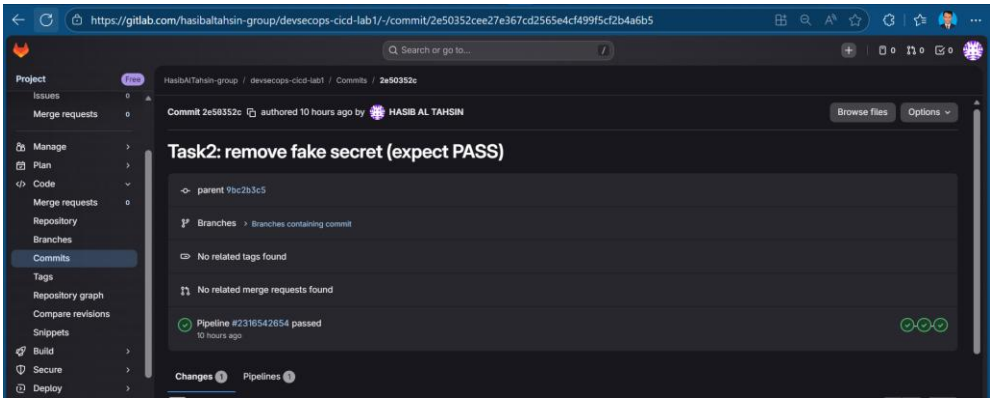
The output *[main 2e50352]* shows that a new commit hash has been created. *1 file changed, 4 deletions(-)* means that four lines inside that file (where the fake secret was) have been effectively deleted. The output confirms that the file has been completely removed from the repository. Here *100644* is the file permission mode, which indicates a normal non-executable file.

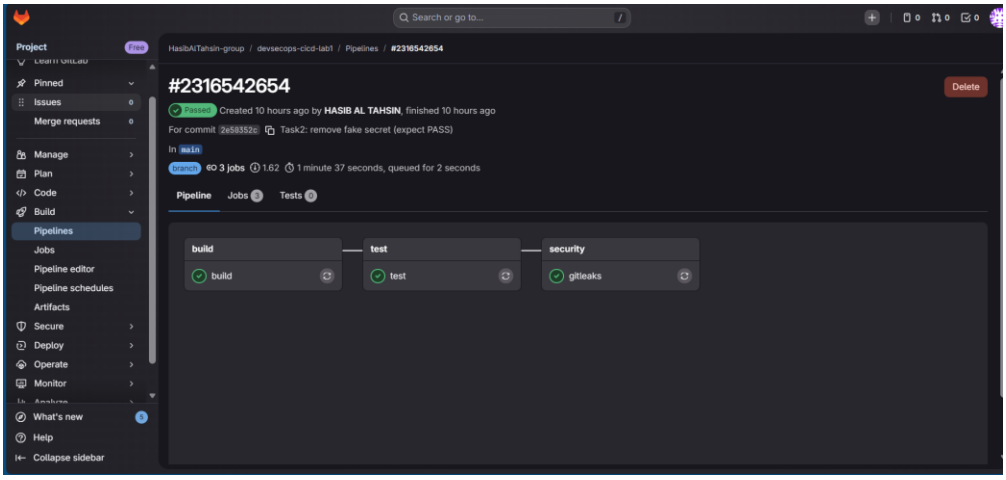
The change is then sent to the main branch of the remote GitLab repository with *git push.*

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 328 bytes | 164.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   9bc2b3c..2e50352  main -> main
```

The push output shows *main -> main*, indicating that the remote main has been updated from local main. Since the pipeline will run only when the CI configuration is pushed to the main branch, a new pipeline is automatically run after this push.

In the GitLab UI, going to the Pipelines section from CI/CD, you can see that the *gitleaks* job has not failed again. This is because there are no more hardcoded secrets in the codebase.





As a result, the security stage passed and the entire pipeline showed green (PASS). This proved that the secret scanning gate is working effectively so that the secret is passed. This is an important proof of DevSecOps control validation.

Task 3: SAST (Static Application Security Testing)

Add a SAST job that analyzes the source code after the build. Configure the tool for your language and define a quality gate that fails the pipeline when critical or high-severity issues are found.

Export the results as a report artifact. Optionally, fix one finding and demonstrate a successful run.

Tool choice: SonarQube, Semgrep, or CodeQL (or Checkmarx, Fortify, Bandit, ESLint security).

Solution:

Adding Semgrep SAST job:

Open the *.gitlab-ci.yml* file and add a job named *semgrep_sast.*

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano .gitlab-ci.yml
```

Here, by giving *stage: security*, the job runs in the security stage after build/test.

```
 GNU nano 7.2                                  .gitlab-ci.yml *
semgrep_sast:
  stage: security
  image: returntocorp/semgrep:latest
  script:
    - mkdir -p reports
    - semgrep scan --config=p/owasp-top-ten --json --output reports/semgrep.json
    - |
      python - << 'PY'
      import json, sys
      data=json.load(open("reports/semgrep.json"))
      results=data.get("results",[])
      high=[]
      for r in results:
        sev=str(r.get("extra",{}).get("severity","")).upper()
        if sev in ("HIGH","CRITICAL","ERROR"):
          high.append(r)
      print("Total findings:", len(results))
      print("High/Critical findings:", len(high))
      if len(high)>0:
        sys.exit(1)
      PY
  artifacts:
    when: always
    paths:
      - reports/semgrep.json
    expire_in: 14 days

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q
```

Using *image: returntocorp/semgrep:latest, semgrep* is available from the Docker image even if you do not install *semgrep* separately inside the runner. In the script, first a folder is created to store reports with *mkdir -p reports*, *then semgrep scan --config=p/owasp-top-ten --json --output reports/semgrep.json* is run. Then the following embedded

Python part is added so that it reads the severity from the report and counts *"High/Critical/ERROR*", and if there is at least 1 *High/Critical*, then the job fails with *sys.exit(1).* Finally, *reports/semgrep.json* is placed in the artifacts section, and when: always is given. As a result, the report will be kept regardless of pass/fail, so that it can be submitted as evidence.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

Then I did git add, commit and push.

commit:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task3: add Semgrep SAST + High/Critical quality gate + artifact"
[main 7a4b24c] Task3: add Semgrep SAST + High/Critical quality gate + artifact
 1 file changed, 28 insertions(+)
```

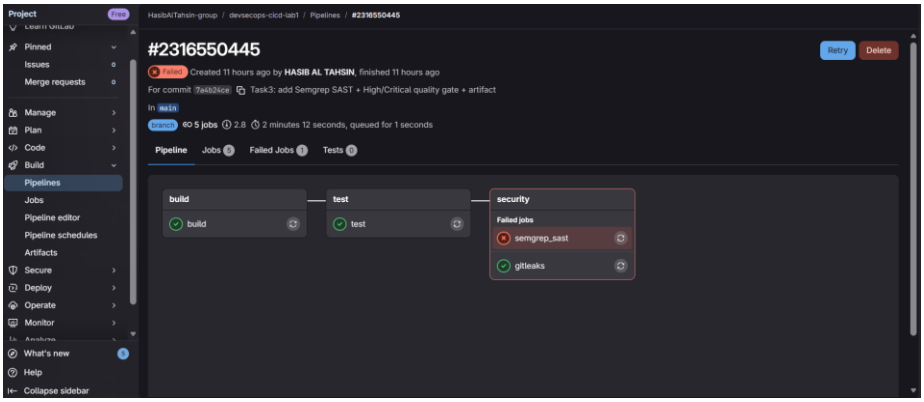
push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 707 bytes | 707.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   2e50352..7a4b24c  main -> main
```

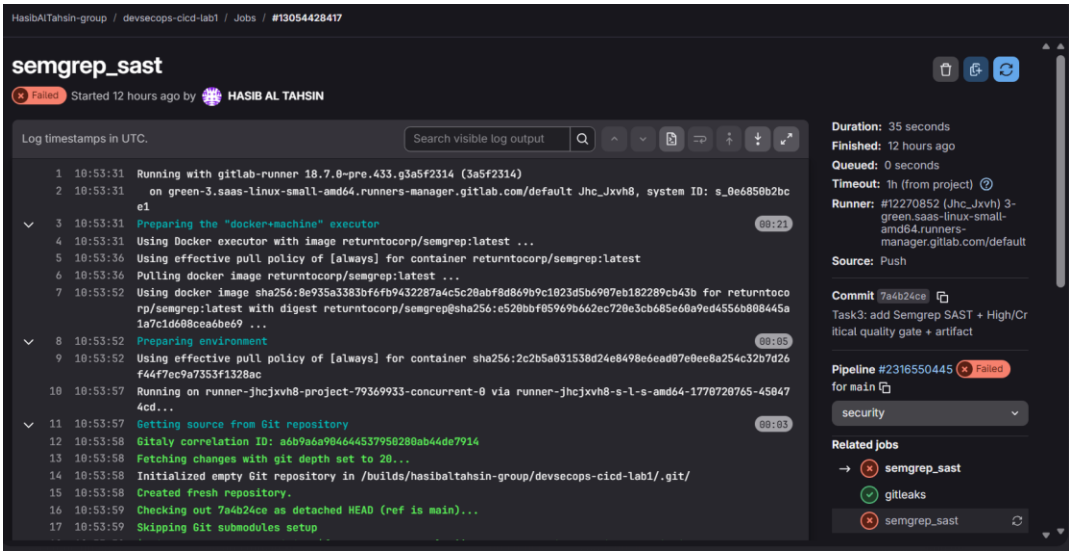
The commit output shows *28 insertions (+),* meaning the job was added and the pipeline ran again according to the main branch trigger as soon as it was pushed.

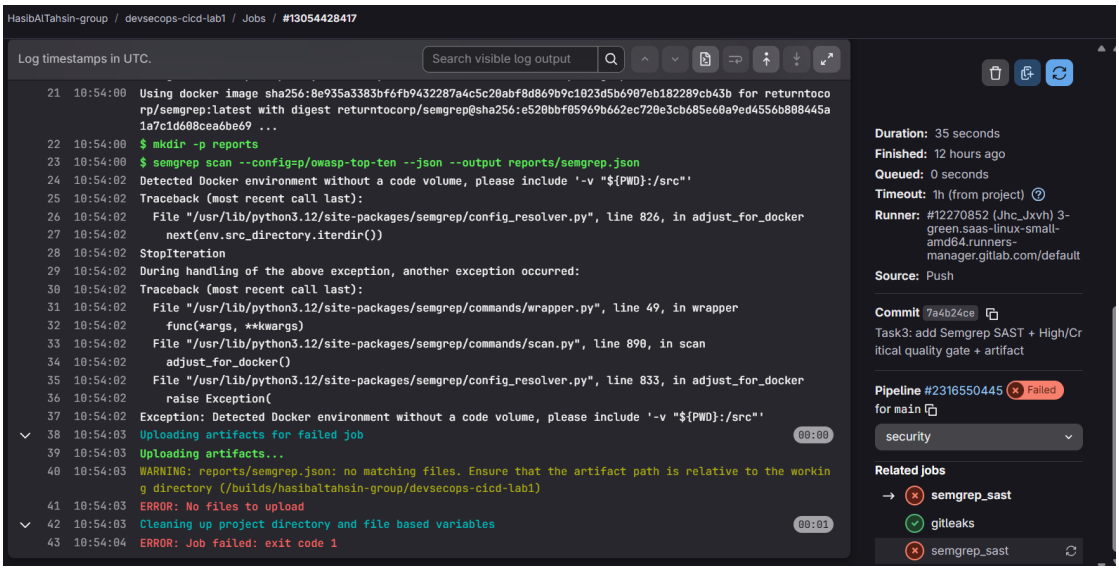
Evidence:

What is shown in the pipeline log is not actually a failure due to a "*vulnerability detected in the code*", but a tool/environment issue while running *semgrep*.



At the beginning of the log, the runner environment is set, the *returntocorp/semgrep:latest* image is pulled using the Docker executor, and the repo is checked out.





The problem started with this line: After running *semgrep scan ... semgrep said: "Detected Docker environment without a code volume, please include '-v "${PWD}:/src"'".* This means that *semgrep* is running from within the image in a way where it has the expectation of "*mounting the volume and viewing the code*", but even though the code workspace is in the docker executor of GitLab Runner, *semgrep* is confused by not finding it inside its */src volume-layout*. Then *semgrep* itself throws an error with *StopIteration* in the traceback and finally Exception: Detected Docker environment without a *code volume... . Result:* The job exit code ends with 1, resulting in *semgrep_sast* job failure. Since *semgrep* scan crashed first, the *reports/semgrep.json* file was not created. So at the artifacts upload stage, *GitLab says: WARNING: reports/semgrep.json: no matching files* and *finally ERROR: No files to upload*. That is, both job fail + artifact missing came because the scan output was not created.

Fixing for PASS:

To overcome the above issue, the *semgrep* job has been changed to be more realistic, meaning instead of using a separate *semgrep* container, *the python:3.12-slim image* has been installed inside it using *pip*. This no longer has the Docker volume detection issue, as *semgrep* can now read repo files inside the workspace of that job container as a normal CLI tool.

```
semgrep_sast:
  stage: security
  image: python:3.12-slim
  cache:
    key: ${CI_COMMIT_REF_SLUG}-pip
    paths:
      - .pip-cache/
  script:
    - mkdir -p reports .pip-cache
    - python -V
    - pip install --cache-dir .pip-cache semgrep
    - semgrep scan --config p/owasp-top-ten --metrics=off --json --output reports/semgrep.json .
    - |
      python - <<'PY'
      import json, sys
      data = json.load(open("reports/semgrep.json"))
      results = data.get("results", []) or []
      high = []
      for r in results:
        sev = str(r.get("extra", {}).get("severity", "")).upper()
```

```
      if sev in ("HIGH", "CRITICAL", "ERROR"):
        high.append(r)
    print("Total findings:", len(results))
    print("High/Critical findings:", len(high))
    sys.exit(1 if len(high) > 0 else 0)
    PY
artifacts:
  when: always
  paths:
    - reports/semgrep.json
  expire_in: 14 days
```

The new job has added a cache for *.pip-cache/,* so that pip dependency download time is reduced even if the pipeline is run repeatedly. The folder has been created in the script by *mkdir -p reports .pip-cache*. The runtime has been verified with *python -V.* Then semgrep has been installed with *pip install* --cache-dir .pip-cache semgrep. Then the scan command has been told to scan this directory, and sending metrics has been stopped with *--metrics=off*.

Then git add, commit, push has been done as before, and as a result the pipeline has been run again.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

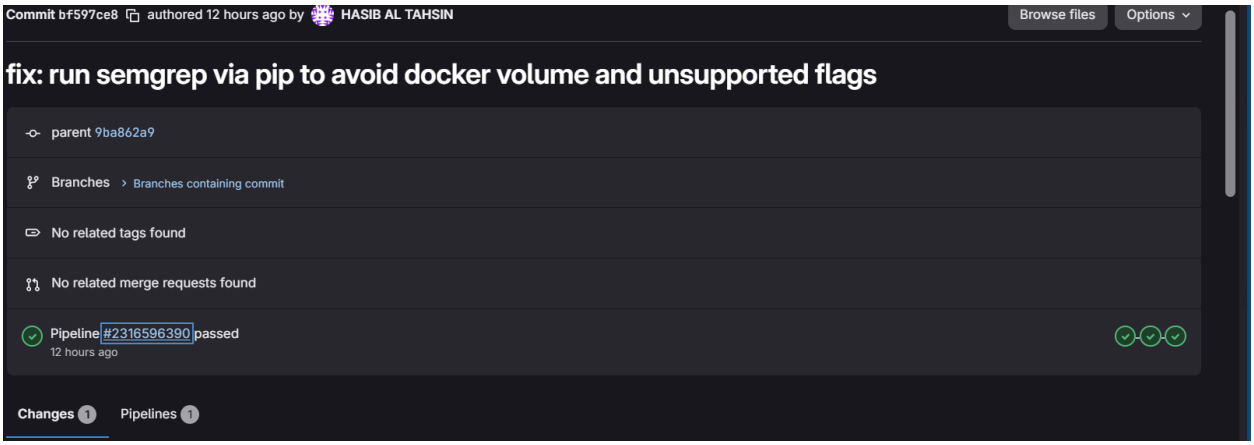
commit:

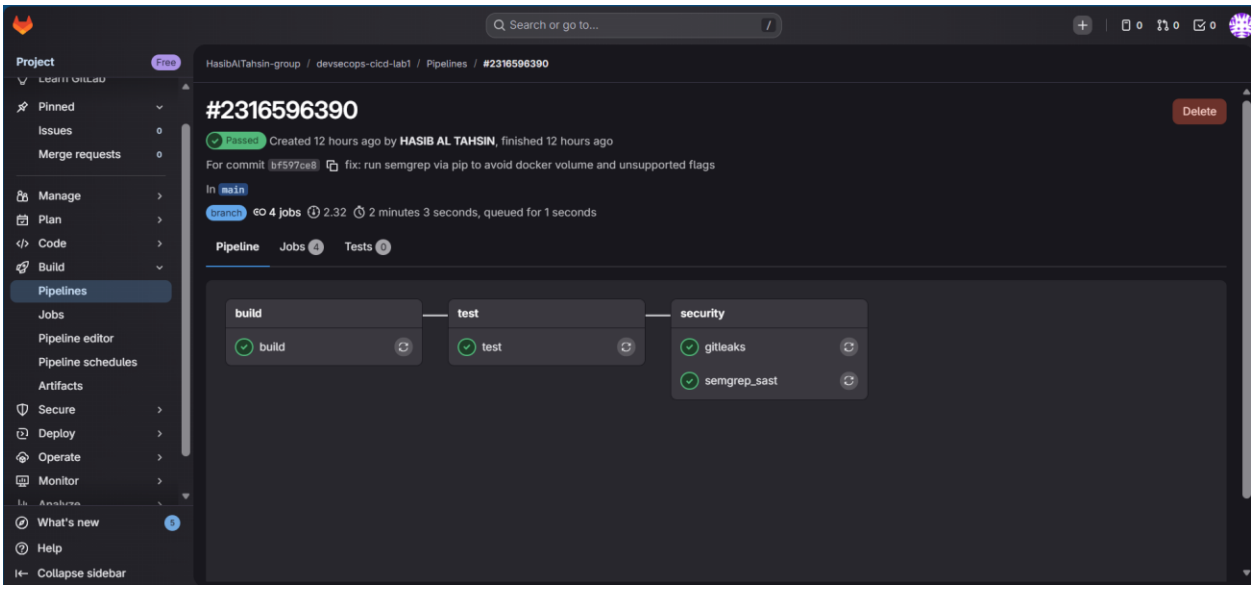
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: run semgrep via pip to avoid docker volume and unsupported flags"
[main bf597ce] fix: run semgrep via pip to avoid docker volume and unsupported flags
 1 file changed, 15 insertions(+), 13 deletions(-)
```

push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 594 bytes | 594.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   9ba862a..bf597ce  main -> main
```

Later, the implementation was fixed and the pipeline was stabilized and found to be stable.





Task 4: SCA / Dependency Scanning

Add a dependency-scanning stage that runs on the same codebase and lockfiles (or manifests) used for the build. The tool should report known vulnerabilities (CVEs) and, if supported, license issues. Define a clear policy (e.g. fail the pipeline when any dependency has a critical or high-severity vulnerability; optionally warn on medium) and implement it in the pipeline. Document how a developer reads the report and remediates findings (e.g. by upgrading a dependency or applying a patch). In the deliverable, show how the pipeline behaves when a vulnerable dependency is present versus when it is fixed or waived.

Tool choice: Snyk, Dependabot, or OWASP Dependency-Check (or Renovate, Trivy, WhiteSource). Same lockfiles/manifests as build.

Solution:

Adding the Trivy SCA job:

I opened *.gitlab-ci.yml* and added a job called *trivy_sca,* which runs on *stage: security.*

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano .gitlab-ci.yml
```

```
trivy_sca:
  stage: security
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  script:
    - mkdir -p reports
    - trivy fs --scanners vuln --severity MEDIUM --exit-code 0 --format table --output reports/trivy-medium.txt . || true
    - trivy fs --scanners vuln,license --severity HIGH,CRITICAL --exit-code 1 --format json --output reports/trivy-sca.json .
  artifacts:
    when: always
    paths:
      - reports/trivy-sca.json
      - reports/trivy-medium.txt
    expire_in: 14 days
```

Here, the *aquasec/trivy:latest* image is used so that Trivy does not have to be installed separately inside the runner. *entrypoint: [""]* is given because many container images have default entrypoints that cause problems in running the CI script's shell commands properly; if the entrypoint is left blank, the GitLab runner can run the following commands normally through the shell.

As the first task of the script, I did *mkdir -p reports* so that there is space to store the report file. Then, two scans were run according to the policy. The first scan was *trivy fs --scanners vuln --severity MEDIUM --exit-code 0 --format table --output reports/trivy-medium.txt* .This is basically an informational report, MEDIUM severity, but since --exit-code 0 is present, the pipeline will not fail. Here *|| true* is kept so that even if the command

warning/issue is given for some reason, the job does not stop and the report is definitely generated. The second scan is the *main gate: trivy fs --scanners vuln,license --severity HIGH,CRITICAL --exit-code 1 --format json --output reports/trivy-sca.json ..* Here, both vulnerability and license are being viewed with *--scanners vuln,license. --severity HIGH,CRITICAL* means that only serious risks are being captured in the gate. And the most important *--exit-code 1* means that if a High/Critical vulnerability or related issue is detected, the job will give exit code 1 and the pipeline will fail. *JSON* output is kept because CVE, package name, severity, fixed version, etc. can be easily extracted from JSON later.

I have kept when: always in the artifacts section, so that *reports/trivy-sca.json* and *reports/trivy-medium.txt* can be downloaded in both pass or fail conditions. This way, the evidence/audit trail is maintained. Then I added and *committed: fix: trivy entrypoint* and add SCA job with artifacts and git pushed.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

commit:

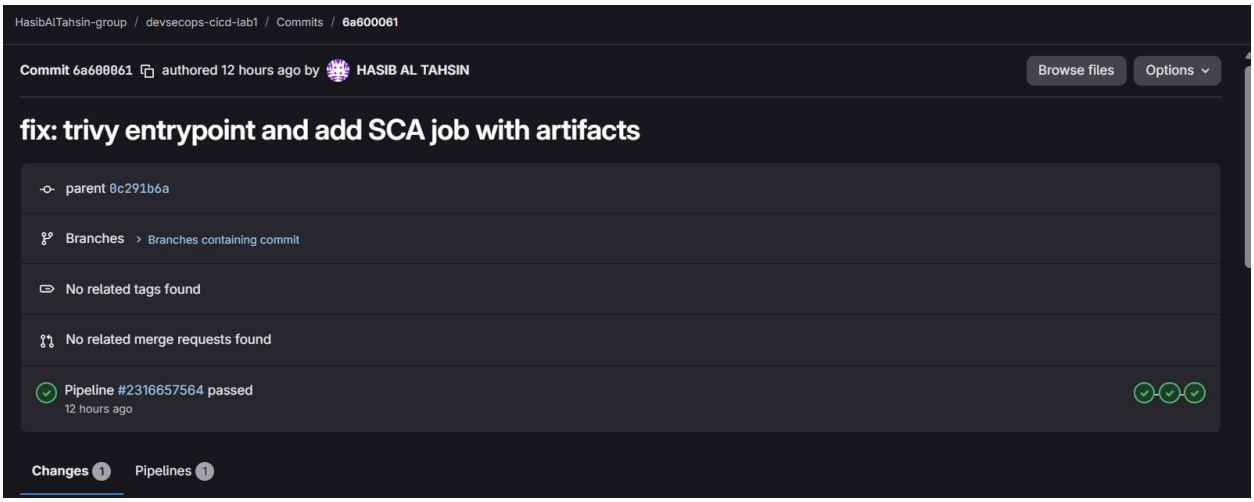
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: trivy entrypoint and add SCA job with artifacts"
[main 6a60006] fix: trivy entrypoint and add SCA job with artifacts
 1 file changed, 7 insertions(+), 6 deletions(-)
```

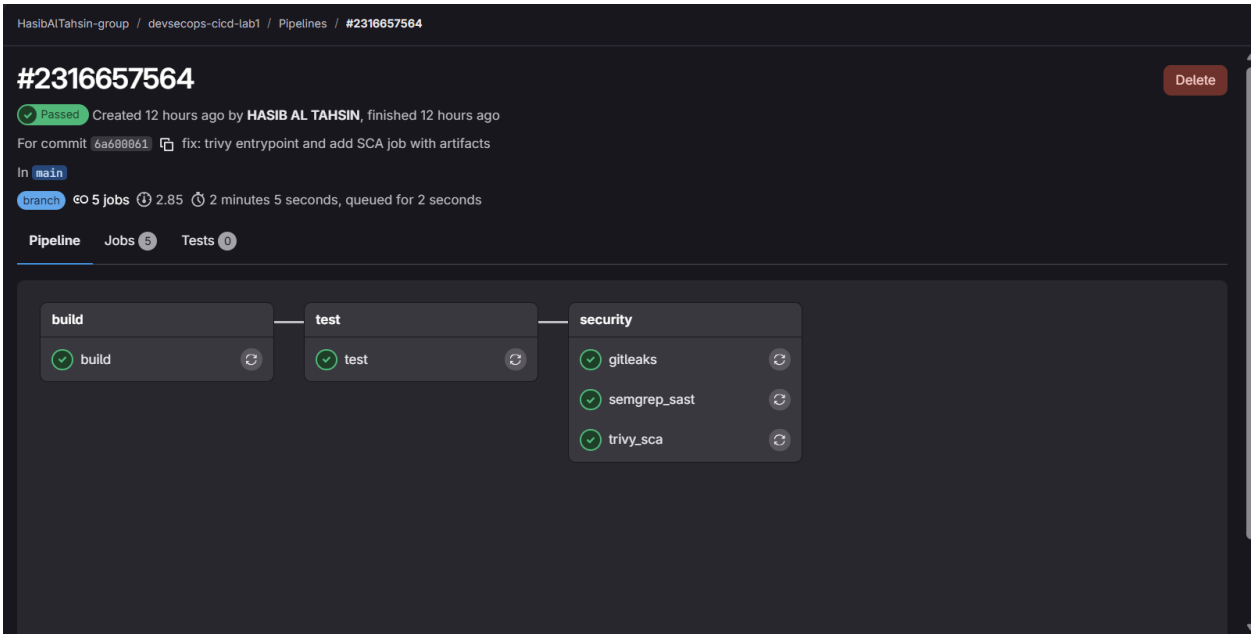
push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 430 bytes | 430.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   0c291b6..6a60006  main -> main
```

The commit output shows that *.gitlab-ci.yml* has been updated and the pipeline has been triggered because *main -> main* in the push output, meaning the job has actually been added to the pipeline.

Pipelines:





Proof: Showing by adding vulnerable dependency:

In this part I intentionally added a vulnerable dependency to prove that *Trivy* gate works. I installed an older version of *lodash* with the command *npm i lodash@4.17.20*.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm i lodash@4.17.20

added 1 package, and audited 381 packages in 5s

70 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

The output shows 1 high severity vulnerability that *npm* itself has detected a high risk issue, and *npm audit* has suggested a fix. That is, there is evidence locally that this is a vulnerable version.

Then I ran *npm audit* and took a detailed report. Here it can be seen that there is a *High severity* issue in the *lodash <=4.17.21* range and multiple advisory links are shown.



Then I staged *package.json* and *package-lock.json* and *committed: Task4 proof: add vulnerable dependency (expect trivy_sca FAIL).*

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add package.json package-lock.json
```

The commit output shows that 2 files have changed (because the *lockfile* is updated when a dependency is added).

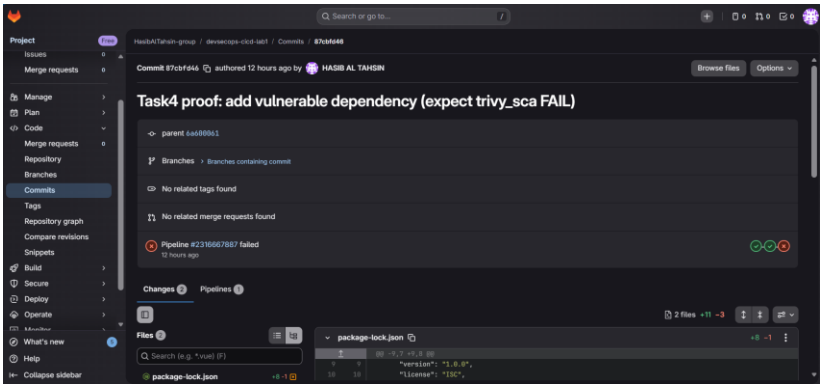
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task4 proof: add vulnerable dependency (expect trivy_sca FAIL)"
[main 87cbfd4] Task4 proof: add vulnerable dependency (expect trivy_sca FAIL)
 2 files changed, 11 insertions(+), 3 deletions(-)
```

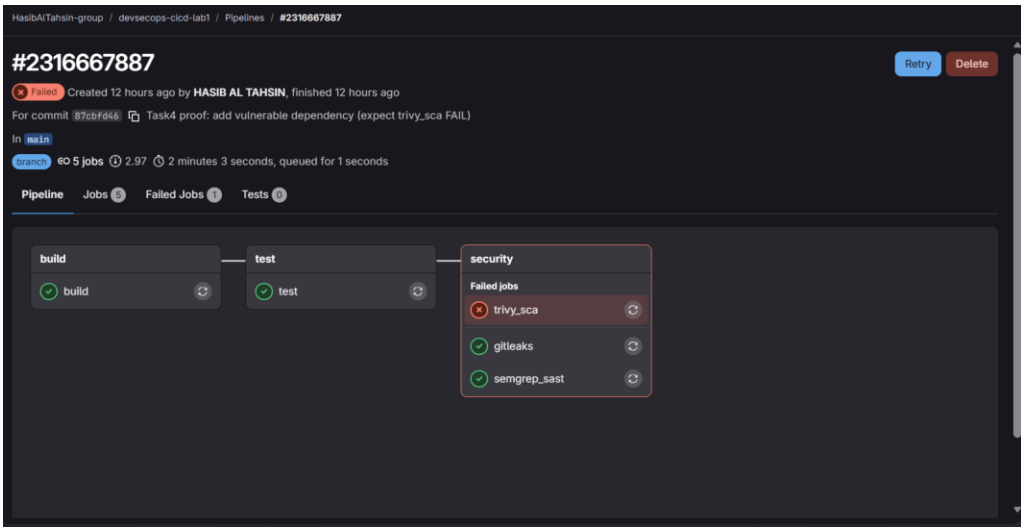
After git push, the new pipeline ran and as evidence I saw that the *trivy_sca job FAILED* and *trivy-sca.json* was created as an artifact.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 631 bytes | 631.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   6a60006..87cbfd4  main -> main
```

This part is the *main evidence: Trivy* gate is stopping the pipeline if there is a vulnerable dependency.

Evidence: trivy_sca FAIL + artifact json





Fix: Update dependency and show:

Then as a remediation I updated *lodash* with *npm i lodash@latest*.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ npm i lodash@latest

changed 1 package, and audited 381 packages in 4s

70 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

The output found 0 vulnerabilities shows that *npm audit* is not finding any more vulnerabilities after the update. That is, the risk has been eliminated on the local side.

Then I added *package.json* and *package-lock.json* again and *committed: fix: upgrade lodash* to remediate SCA and *git push origin main.*

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add package.json package-lock.json
```

commit:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: upgrade lodash to remediate SCA"
[main 0014aca] fix: upgrade lodash to remediate SCA
 2 files changed, 5 insertions(+), 5 deletions(-)
```

The pipeline was triggered again due to the push, and this time since the vulnerable package is no longer there, the *trivy_sca* job passed. This is the "*vuln vs fixed proof*".

push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 529 bytes | 529.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   87cbfd4..0014aca  main -> main
```

That is, first fail, then fix and pass-the entire remediation cycle has been successfully completed.

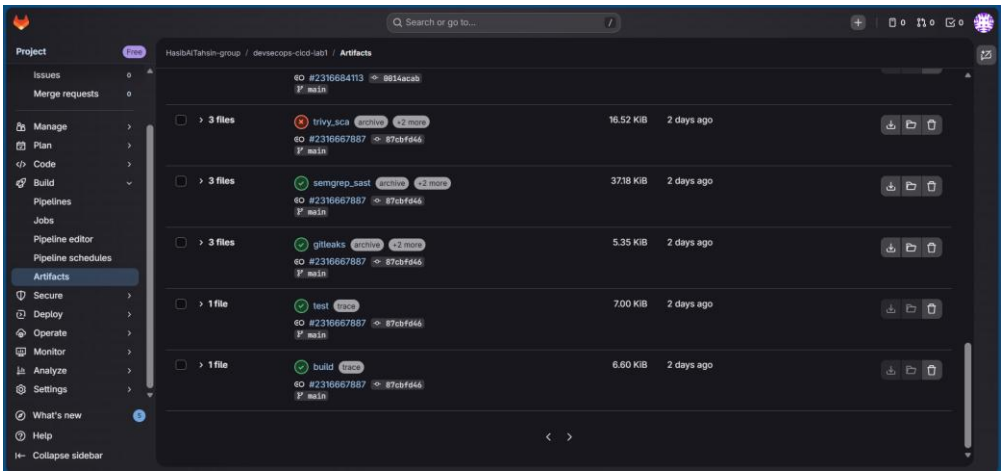
Developer remediation doc

After the trivy_sca stage runs, the pipeline produces an artifact:

reports/trivy-sca.json

A developer downloads this file from:

Artifacts



Inside the JSON report, the developer checks:

Key Fields to Analyze

Field	Meaning
PkgName	Vulnerable package name
InstalledVersion	Currently installed version
FixedVersion	Secure version (if available)
Severity	LOW / MEDIUM / HIGH / CRITICAL
CweIDs	Type of weakness (e.g., CWE-94)
CVSS.V3Score	Risk score

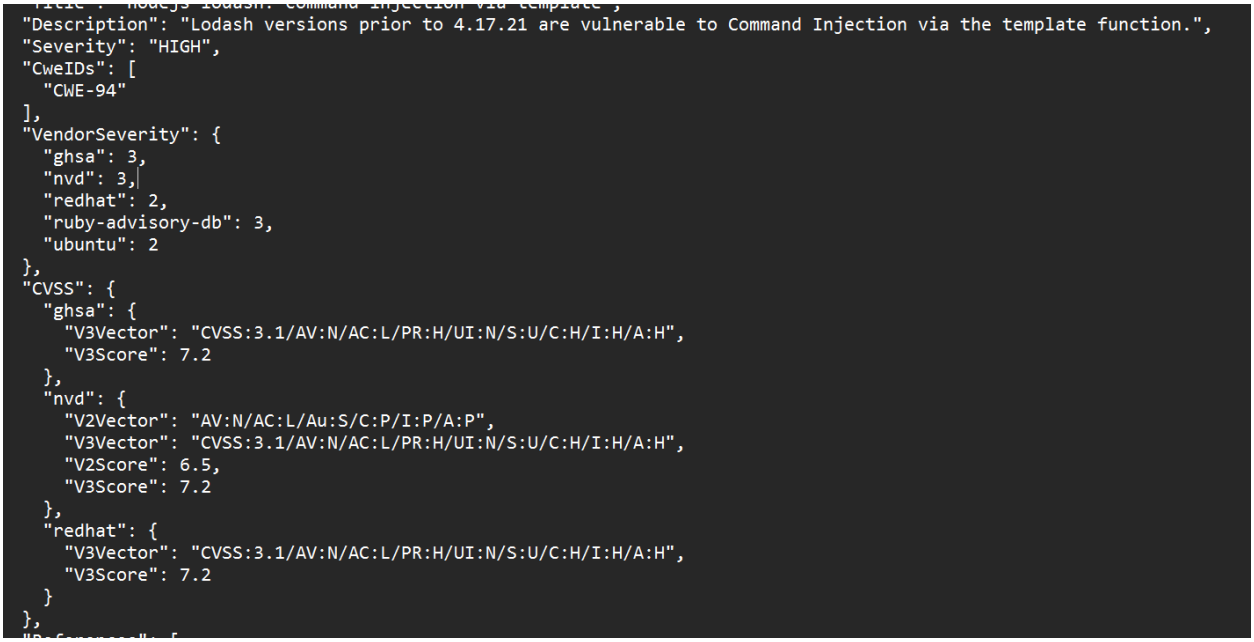
Finding

Severity: HIGH

CweIDs: CWE-94

CVSS Score: 7.2

Type: npm



This indicates:

- The vulnerability is HIGH severity.
- It relates to Code Injection (CWE-94).
- The risk score is 7.2 (High).
- It exists in an npm dependency.

Because the pipeline policy is configured to:

Fail on HIGH or CRITICAL vulnerabilities

The pipeline stops at the trivy_sca stage.

Task 5: Container and Image Security (Optional bonus task)

If the application is containerized, add a stage that builds a container image and tags it with a traceable identifier. Scan the image for vulnerabilities and fail the pipeline on critical or high findings.

Tool choice: Trivy, Grype, or Snyk Container (or Clair, Anchore). Optionally cosign / in-toto for signing.

Solution:

The *GitLab SaaS runner* does not always enable privileged mode, especially when using Docker-in-Docker (*dind*), which can cause jobs to fail in some environments. However, it has been added to the container security pipeline, allowing for application image builds and future image-level vulnerability scans.

I added a job called *docker_build_and_scan* to *.gitlab-ci.yml*, which runs in stage: container.

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ nano .gitlab-ci.yml
```

```
    d = json.load(open(p))
    hit = 0
    for r in (d.get("Results") or []):
        for v in (r.get("Vulnerabilities") or []):
            if v.get("Severity") in ("HIGH", "CRITICAL"):
                hit += 1

    print("HIGH/CRITICAL vulnerabilities:", hit)
    sys.exit(1 if hit > 0 else 0)
    PY
artifacts:
  when: always
  paths:
    - reports/trivy-image.json
  expire_in: 14 days


^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Loc
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go
```

```
 GNU nano 7.2                                  .gitlab-ci.yml *
docker_build_and_scan:
  stage: container
  image: docker:27
  services:
    - name: docker:27-dind
      command: ["--tls=false"]
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""
  script:
    - mkdir -p reports
    - export IMAGE_TAG="${CI_COMMIT_SHORT_SHA}"
    - docker version
    - docker build -t "devsecops-lab:${IMAGE_TAG}" .

    - docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v "$PWD:/work" -w /work aquasec/trivy:latest image --format json --output reports/trivy-image.

    - |
      docker run --rm -v "$PWD:/work" -w /work python:3.12-slim python - <<'PY'
      import json, os, sys
      p = "reports/trivy-image.json"
      if not os.path.exists(p):
          print("trivy-image.json missing")
          sys.exit(1)
```

```
  - |
    docker run --rm -v "$PWD:/work" -w /work python:3.12-slim python - <<'PY'
    import json, os, sys
    p = "reports/trivy-image.json"
    if not os.path.exists(p):
        print("trivy-image.json missing")
        sys.exit(1)

    d = json.load(open(p))
    hit = 0
    for r in (d.get("Results") or []):
        for v in (r.get("Vulnerabilities") or []):
            if v.get("Severity") in ("HIGH", "CRITICAL"):
                hit += 1

    print("HIGH/CRITICAL vulnerabilities:", hit)
    sys.exit(1 if hit > 0 else 0)
    PY
artifacts:
  when: always
  paths:
    - reports/trivy-image.json
  expire_in: 14 days
```

Here, *image: docker:27* is used, that is, Docker CLI is available inside the job container. Also, *docker:27-dind* has been added to *services:*. TLS has been disabled with command: *["--tls=false"],* because Docker daemon is being used on local network in CI environment.

In variables section, *DOCKER_HOST: tcp://docker:2375* has been set, so that Docker CLI can connect to that *dind* service. *DOCKER_TLS_CERTDIR: ""* has been given to disable TLS auto-setup.

In script section, first *mkdir -p reports* has been kept for future scan reports. Then, the image has been tagged according to the commit hash with export *IMAGE_TAG="${CI_COMMIT_SHORT_SHA}".* This creates a different image tag for each commit. Running docker version has ensured that Docker CLI and daemon are communicating properly. Then running *docker build -t "devsecops-lab:${IMAGE_TAG}" .* the image has been built using *Dockerfile* from the current repository.

Then *.gitlab-ci.yml* was added, committed and pushed.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add .gitlab-ci.yml
```

commit:

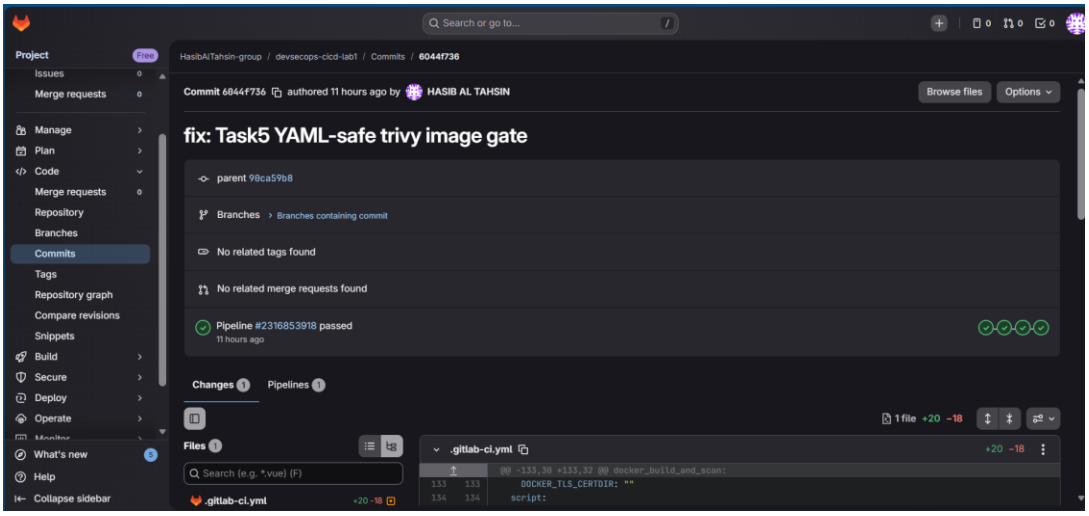
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "fix: Task5 YAML-safe trivy image gate"
[main 6044f73] fix: Task5 YAML-safe trivy image gate
 1 file changed, 20 insertions(+), 18 deletions(-)
```

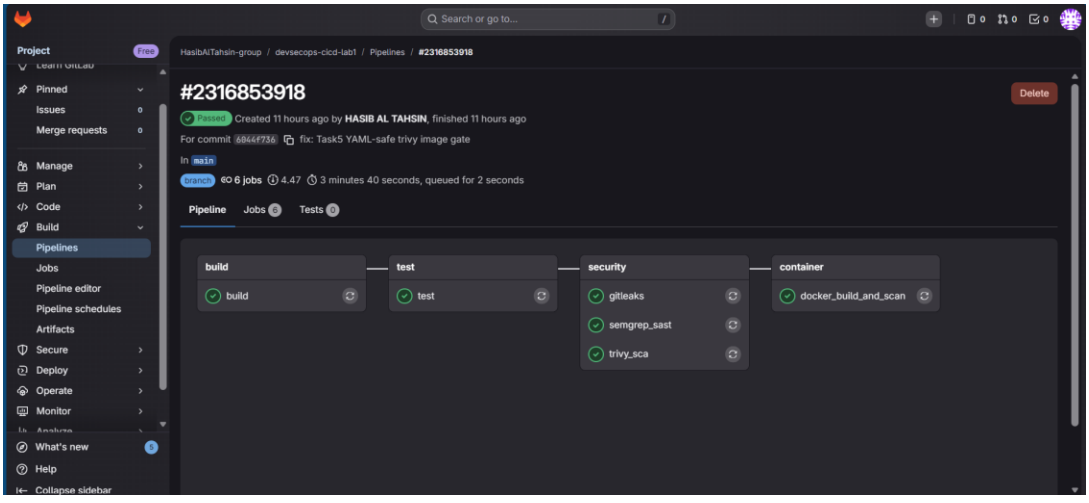
push:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 628 bytes | 628.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   90ca59b..6044f73  main -> main
```

The commit output shows that the *YAML* has changed and the pipeline has been triggered after the push. If the runner was not privileged, this job would have failed. But the build succeeded and the container image was created successfully.

Pipeline:





Task 6: Reporting, Metrics, and Evidence

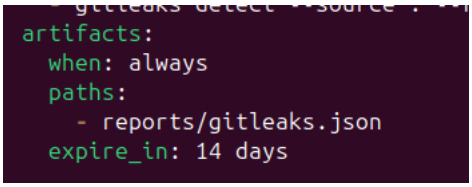
Configure the pipeline to retain security reports as artifacts (SAST, SCA, and optionally container reports). Define at least three metrics that measure security effectiveness, such as vulnerability severity trends, remediation time, or security-related pipeline failures.

Solution:

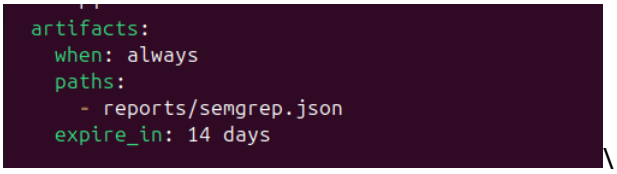
Verifying Artifacts Retention in Security Jobs:

First, *.gitlab-ci.yml* was opened and verified that each security related job had artifacts: when: always . Specifically, it was verified that:

- **Gitleaks**

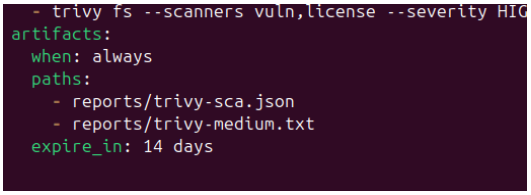


- **semgrep_sast**

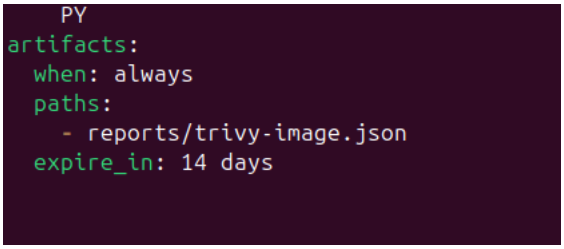


\

- **trivy_sca**



- **docker_build_and_scan**

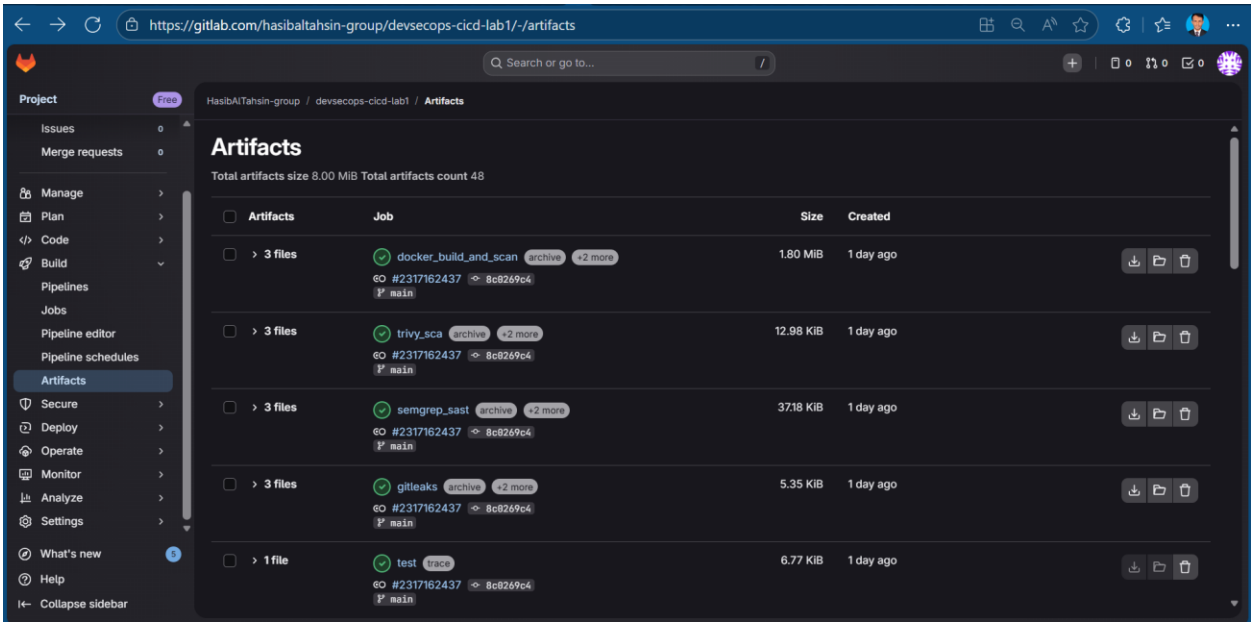


Each job ensures that the scan report is saved as a *reports/*.json* file and has an *expire_in: 14 days.* Using when: always means that the report remains as an artifact whether the job passes or fails.

Verify that artifacts are actually being created:

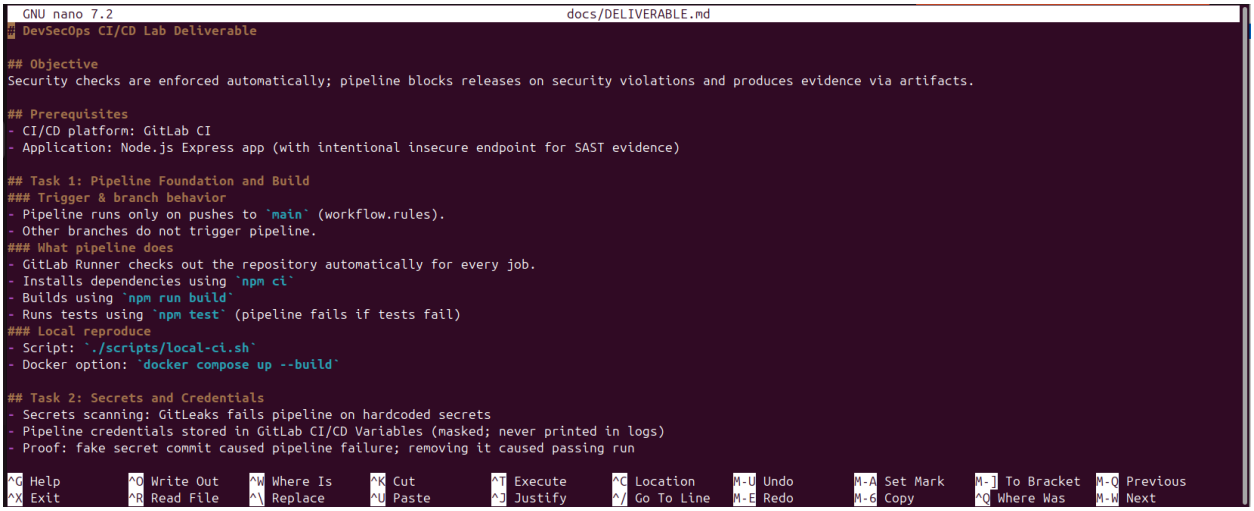
By going to the GitLab UI and going to the *Jobs* > *Artifacts* section, we confirmed that the following reports are being created:

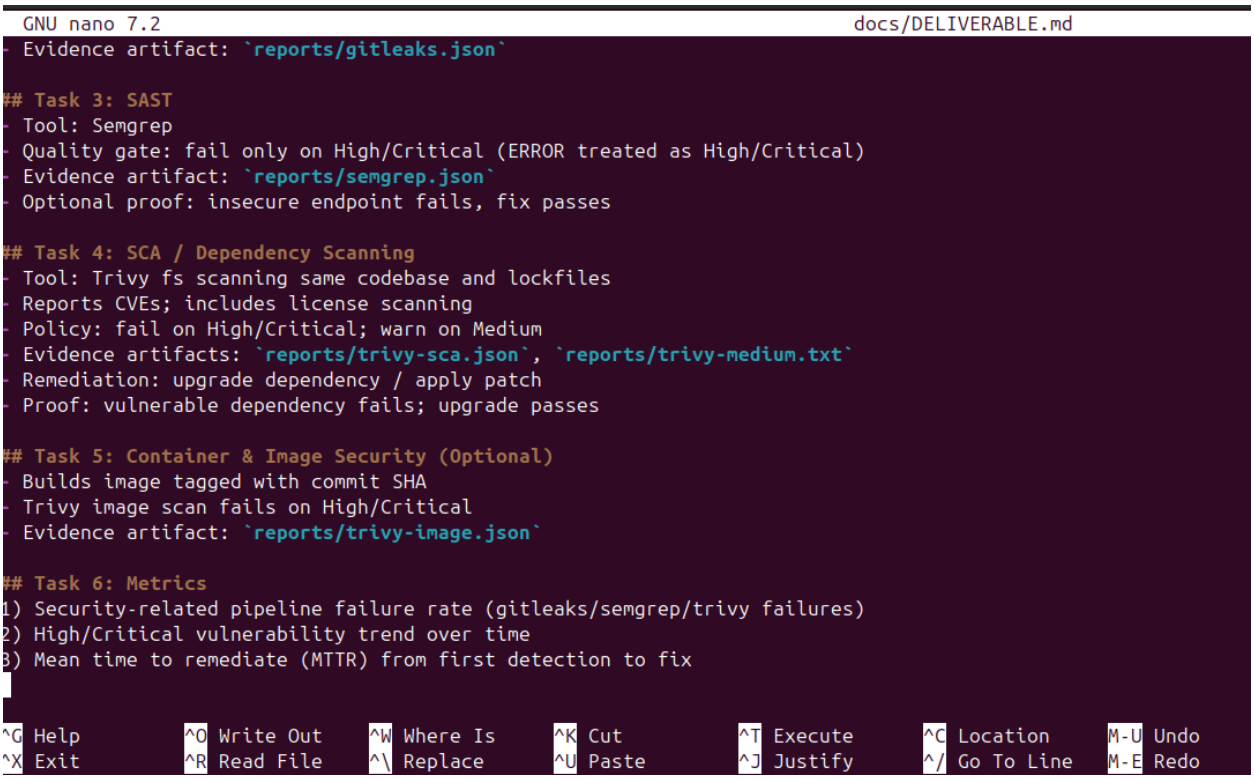
- **reports/gitleaks.json**
- **reports/semgrep.json**
- **reports/trivy-sca.json**
- **reports/trivy-medium.txt**
- **reports/trivy-image.json**



The presence of these files in the artifacts list proves that the pipeline is not just scanning-it is also storing the results.

A *docs/DELIVERABLE.md* file has been created that documents the entire *DevSecOps* lab objective, a brief description of each task, policy, proof, and artifact location.





It clearly states:

- Which tool was used at which stage
- At what severity level is the pipeline failing
- Which report artifacts will be available
- How was the remediation done
- Vulnerability fail vs fixed pass evidence

The file was added to the repository, committed and pushed.

add:

```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git add docs/DELIVERABLE.md
```

commit:

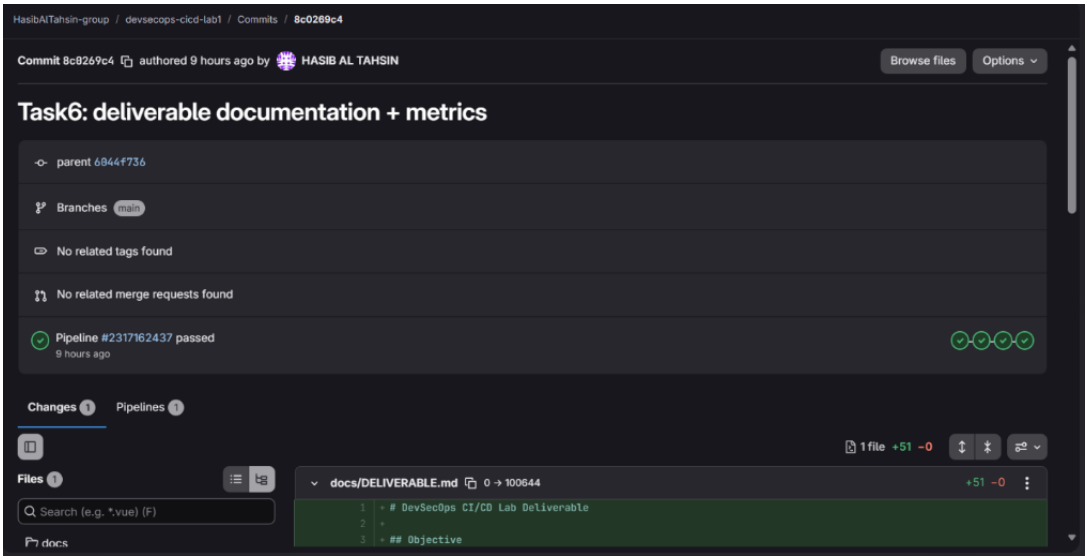
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git commit -m "Task6: deliverable documentation + metrics"
[main 8c0269c] Task6: deliverable documentation + metrics
 1 file changed, 51 insertions(+)
 create mode 100644 docs/DELIVERABLE.md
```

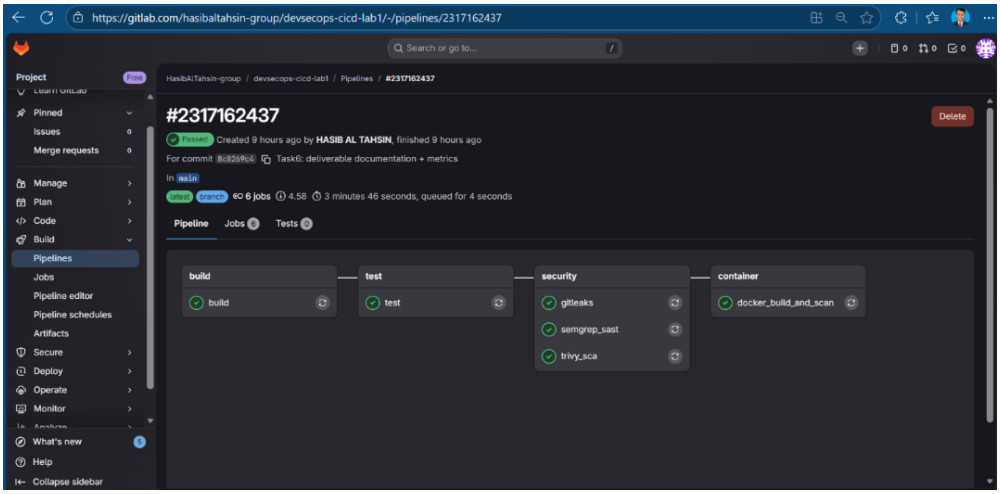
The commit output shows create mode *100644 docs/DELIVERABLE.md*, indicating that the file has been newly added to the repository.

push:

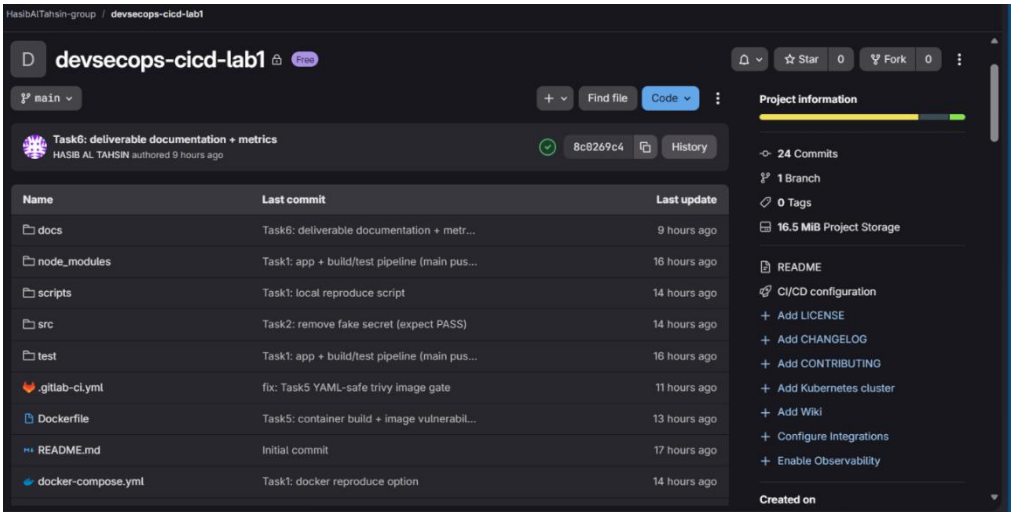
```
hasib@gitlab-24:~/devsecops-cicd-lab1$ git push origin main
Username for 'https://gitlab.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@gitlab.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.30 KiB | 1.30 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/hasibaltahsin-group/devsecops-cicd-lab1.git
   6044f73..8c0269c  main -> main
```

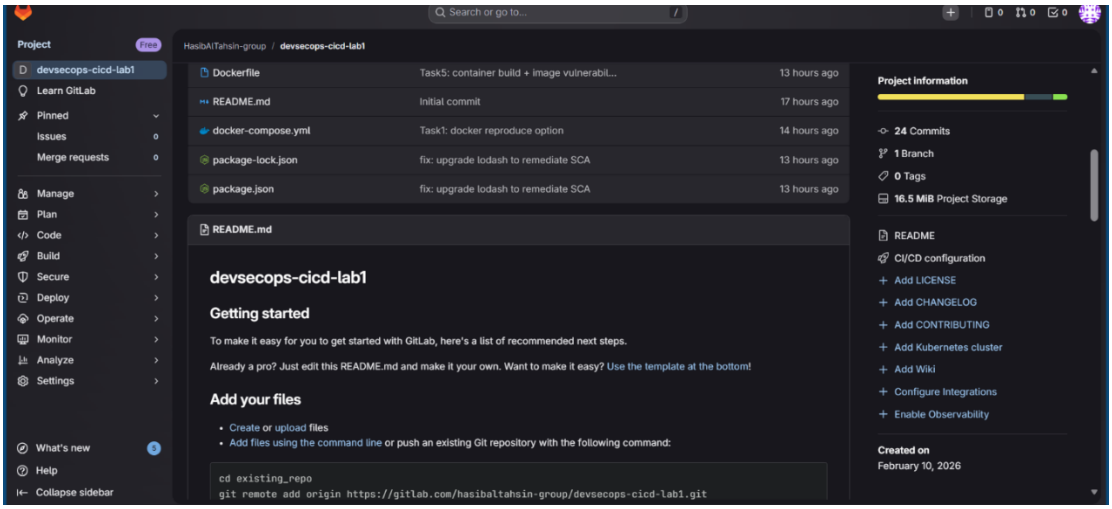
As soon as the push was made, the pipeline ran again, proving that the documentation is version-controlled and traceable.





Project directory:





Metrics:

Here, three measurable security metrics are defined and calculated.

Security-related Pipeline Failure Rate:

The status of each pipeline is monitored by going from Build to Pipelines section. Pipelines that have failed the security stage job (such as *gitleaks, semgrep_sast, trivy_sca, docker_build_and_scan*) are counted as security failures.

Total pipeline runs = 15

Failed due to security reasons = 6

Then failure rate = (6/15)*100 = 40%

This metric shows what percentage of pipelines are blocked due to security issues.

High/Critical Vulnerability Trend:

The number of High/Critical vulnerabilities was calculated by downloading *reports/trivy-sca.json* from Artifacts.

Before adding vulnerable dependency: 7 High/Critical

After upgrading dependency: 0 High/Critical

That is, High/Critical vulnerability has decreased from 7 to 0. This metric shows the improvement of security posture over time.

MTTR (Mean Time To Remediate):

The timestamp of the failed and the next passed pipeline from the pipeline history is compared.

February 10 = *trivy_sca* failed

February 11 = pipeline passed

That is, it took 1 day to remediate.

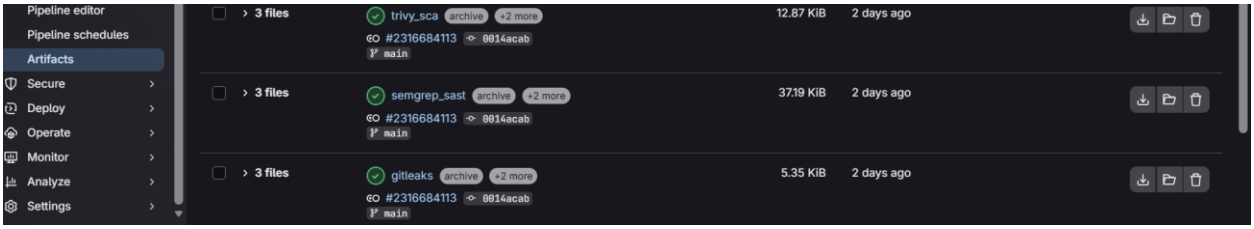
That is, MTTR = 1 day

This metric is important for DevSecOps maturity, because it shows how long it takes to fix a vulnerability after it is detected.

Security Evidence Validation

The main requirement of this task was to preserve security evidence. As seen on the Artifacts page:

- *trivy_sca* archive
- *semgrep_sast* archive
- *gitleaks* archive



This means that each security control is saved in the form of a scan result and an audit trail is maintained.

References:

[1] GitLab, “CI/CD variables,” *GitLab Docs*. Available: https://docs.gitlab.com/ci/variables/

[2] GitLab, “Predefined CI/CD variables reference,” *GitLab Docs*. Available: https://docs.gitlab.com/ci/variables/predefined_variables/

[3] GitLab, “Use Docker to build Docker images,” GitLab Docs. Available: https://docs.gitlab.com/ci/docker/using_docker_build/

[4] npm, “npm-ci,” *npm Docs*. Available: https://docs.npmjs.com/cli/v11/commands/npm-ci/

[5] Jest, “Jest CLI Options,” Jest Documentation. Available: https://jestjs.io/docs/cli

[6] Semgrep, “Run local CLI scans,” *Semgrep Docs*. Available: https://semgrep.dev/docs/for-developers/cli

[7] Semgrep, “CLI reference,” *Semgrep Docs*. Available: https://semgrep.dev/docs/cli-reference

[8] Aqua Security, “Filesystem (Target: filesystem),” *Trivy Documentation*. Available: https://trivy.dev/docs/v0.54/target/filesystem/

[9] Aqua Security, “Filesystem (CLI reference: fs),” *Trivy Documentation*. Available: https://trivy.dev/v0.27.1/docs/references/cli/fs/

[10] Aqua Security, “Licenses,” *Trivy Documentation*. Available: https://trivy.dev/latest/docs/scanner/license/

[11] GitHub, “GHSA-35jh-r3h4-6jhm (lodash command injection advisory),” *GitHub Security Advisories*. Available: https://github.com/advisories/GHSA-35jh-r3h4-6jhm

[12] OWASP Foundation, “OWASP Top Ten,” *OWASP*. Available: https://owasp.org/www-project-top-ten/

[13] Gitleaks, “Gitleaks (detect secrets),” *GitHub repository*. Available: https://github.com/gitleaks/gitleaks